

Regression: Placement and Project Guide

Regression predicts a numeric quantity (or, in logistic regression, a probability). In every model, keep preprocessing and fitting inside a **Pipeline**/cross-validation loop to prevent leakage.

Linear Regression

1. Overview

Linear regression predicts a continuous target as a weighted sum of features. It is the baseline for price, demand, duration, and risk forecasting because it is fast and interpretable.

2. Intuition

Fit the straight line (or hyperplane) that makes predictions as close as possible to observed values: each feature contributes **weight × feature value**.

3. Prerequisites

Vectors/matrices, mean and variance, derivatives, train/test split, Python/NumPy, and MSE.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Coefficients	Feature effects holding other features fixed; a +2 area coefficient adds 2 units per square foot. Ask: correlation is not causation.
Intercept	Prediction when all features are zero; useful only if that point is meaningful.
Residual	y - y_hat ; residual plots reveal nonlinearity/heteroscedasticity.
Assumptions	Linearity, independent errors, roughly constant variance, low multicollinearity, normal residuals mainly for inference.

5. Algorithm / Working Process

Input is feature matrix **X** and numeric target **y**. Split data, impute/encode/scale if needed, fit coefficients by least squares, then output **X_new @ w + b**. Training minimizes squared residuals; inference is one matrix multiplication.

6. Mathematical Foundation

y_hat = Xw + b; minimize **MSE = (1/n) Σ(y_i - y_hat_i)^2**. With a bias column, the closed-form solution is **w = (X^T X)^(-1) X^T y** when invertible; practical libraries use stable SVD/QR. Gradient descent updates **w ← w - η(2/n) X^T (Xw - y)**.

7. Practical Implementation

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2,
random_state=42)
model = make_pipeline(StandardScaler(), LinearRegression()).fit(X_train, y_train)
pred = model.predict(X_test)
print(mean_absolute_error(y_test, pred), root_mean_squared_error(y_test, pred),
r2_score(y_test, pred))
```

8. Code Explanation

The split reserves unseen data. The pipeline scales using training-set statistics only and fits least squares. MAE reports typical absolute error, RMSE penalizes large misses, and R^2 is variance explained relative to predicting the mean.

9. Training / Evaluation

Use a representative random split (time split for time series). Tune feature engineering rather than model capacity. Train R^2 much above test R^2 signals overfitting; low values on both suggest underfitting. Inspect residuals and use CV.

10. Complexity and Cost

Dense least-squares fitting is roughly $O(np^2 + p^3)$ for n rows and p features; prediction is $O(p)$ per row. CPU is normally enough; memory is $O(np)$.

11. Common Use Cases

House prices, sales forecasting, sensor calibration, baseline uplift estimates, and interpretable tabular prediction.

12. Common Mistakes

Scaling/imputing before splitting, interpreting correlation causally, retaining target leakage, using random splits for future prediction, ignoring influential outliers, and reporting only training R^2 .

13. Edge Cases / Limitations

Poor for nonlinear effects, severe outliers, correlated predictors, changing relationships, and extrapolation far beyond training data.

14. Variations

Multiple regression adds features; weighted least squares handles unequal noise; regularized linear models handle multicollinearity. Multiple regression is essential for placements; weighted variants matter in statistics.

15. Related Topics

Polynomial regression adds nonlinear features; Ridge/Lasso regularize weights; logistic regression applies a sigmoid for classification; GLMs generalize the response distribution.

16. Interview Questions

1. **What does OLS minimize?** Sum of squared residuals.
2. **Why square errors?** It is differentiable and punishes large errors.
3. **What is R^2 ?** Improvement over predicting the target mean; it can be negative on test data.
4. **MAE vs RMSE?** MAE is outlier-resistant; RMSE weights large errors more.
5. **Why scale?** Not required for OLS predictions, but helps optimization and comparable coefficients.
6. **What is multicollinearity?** Predictors are correlated, making individual coefficients unstable.
7. **Can linear regression model a curve?** Yes, after nonlinear feature transformation.
8. **Why residual plots?** To check structure left unexplained by the model.
9. **What is adjusted R^2 ?** R^2 penalized for needless predictors.
10. **Why not invert $X^T X$ directly?** It can be numerically unstable or singular.

17. Practice Tasks

Predict a dataset target with NumPy normal equations; build a housing pipeline with CV; compare MAE/RMSE after injecting outliers; diagnose a residual-vs-fitted plot; add one justified interaction feature.

18. Project Ideas

Project	Stack/data/resume value
Rent estimator	Pandas, scikit-learn, local listings; demonstrates EDA and explainability.
Energy-demand forecast	Weather + energy data; shows leakage-aware time split.
Salary analysis	Survey data; shows coefficient interpretation and fairness discussion.

19. Quick Revision

Weighted sum; **MSE**; use for interpretable continuous targets; MAE/RMSE/ R^2 ; traps are leakage, nonlinearity, and outliers. **One-liner:** OLS chooses the hyperplane with minimum squared residual error.

20. Final Cheat Sheet

Definition	I/O	Steps	Metrics	Pros / cons	Best use
Least-squares numeric predictor	features → number	preprocess, fit, predict	MAE, RMSE, R^2	fast/interpretable; linear/outlier-sensitive	strong baseline, explanation

Polynomial Regression

1. Overview

Polynomial regression is linear regression on transformed features such as x^2 and x^3 , so it captures smooth curves in price, physics, and response modeling.

2. Intuition

A straight ruler cannot follow a U-shaped relationship; adding a squared feature lets a linear solver fit a parabola.

3. Prerequisites

Linear regression, feature engineering, bias-variance trade-off, scaling, and cross-validation.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Degree	Highest exponent; degree 2 adds curvature. Higher degree raises variance.
Interactions	$x_1 * x_2$ captures combined effects; degree 2 can create it automatically.
Overfitting	A degree-15 curve can interpolate noise; choose degree by CV.

5. Algorithm / Working Process

Transform X into powers/interactions, optionally scale, then fit ordinary or regularized linear regression. At inference, apply the identical transformation before prediction.

6. Mathematical Foundation

For one feature, $\hat{y} = b_0 + b_1x + b_2x^2 + \dots + b_dx^d$; minimize the same MSE. It remains linear in **coefficients**, not necessarily in input x .

7. Practical Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge

# Ridge prevents high-degree coefficients from exploding.
model = make_pipeline(PolynomialFeatures(degree=3, include_bias=False),
                      StandardScaler(), Ridge(alpha=1.0))
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

8. Code Explanation

PolynomialFeatures creates powers and interactions; scaling follows expansion because magnitudes differ sharply. Ridge is a prudent default when feature count grows.

9. Training / Evaluation

Use a pipeline and **GridSearchCV** over degree and alpha. Plot validation error: decreasing training error with rising validation error is overfitting. Hold out an extrapolation region if that matters.

10. Complexity and Cost

With p inputs and degree d , generated features are $C(p+d, d)-1$, which can explode. Fit and memory costs follow the expanded feature count; CPU suffices for modest tabular data.

11. Common Use Cases

Calibration curves, dose-response, demand curves, engineered scientific features, and nonlinear tabular baselines.

12. Common Mistakes

Using too high a degree, forgetting scaling, transforming test data differently, trusting implausible extrapolation, and using polynomial expansion on high-dimensional data.

13. Edge Cases / Limitations

Global polynomials oscillate at boundaries and extrapolate badly. Splines, trees, or kernels often suit local irregular patterns better.

14. Variations

Interaction-only features reduce growth; spline regression uses piecewise polynomials; Ridge polynomial regression is the practical placement variant. Splines are valuable in projects/statistics.

15. Related Topics

Basis expansion connects to splines and kernels; Ridge controls expansion variance; kernel methods implicitly create rich nonlinear features.

16. Interview Questions

1. **Is it a linear model?** Linear in coefficients, yes.
2. **Why scale?** Powers create drastically different ranges.
3. **How choose degree?** Cross-validation.
4. **What is an interaction?** Product of two feature values.
5. **Why overfit?** Feature count and flexibility grow rapidly.
6. **Can it extrapolate safely?** Usually no.
7. **Why Ridge?** Stabilizes correlated polynomial terms.
8. **Degree 1 equals?** Linear regression.
9. **Alternative to global polynomial?** Splines.
10. **Main complexity risk?** Combinatorial feature explosion.

17. Practice Tasks

Fit degrees 1–10 and plot CV error; add interactions to a housing model; compare raw versus scaled features; inspect boundary predictions; replace the polynomial with splines.

18. Project Ideas

Project	Stack/data/resume value
Vehicle fuel curve	scikit-learn, Auto MPG; nonlinear EDA.
Temperature calibration	NumPy, sensor readings; interpretable curve fitting.
Marketing response	Pandas, advertising data; interactions and CV.

19. Quick Revision

Linear regression on powers; $y=b_0+b_1x+b_2x^2$; choose degree by CV; RMSE/ R^2 ; trap: high degree and extrapolation. **One-liner:** nonlinear feature map, linear coefficient fit.

20. Final Cheat Sheet

Definition	I/O	Key hyperparameters	Pros / cons	Best use
Regression on polynomial features	features → number	degree, Ridge alpha	smooth curves; feature explosion	low-dimensional smooth relationships

Ridge Regression

1. Overview

Ridge is linear regression with L2 regularization. It improves stability when predictors are many or correlated, common in wide tabular and text features.

2. Intuition

OLS may give huge opposing weights to similar features. Ridge charges a cost for large weights, spreading influence more stably.

3. Prerequisites

OLS, MSE, feature scaling, multicollinearity, and cross-validation.

4. Core Concepts

Subtopic	Meaning, example, interview angle
L2 penalty	Adds $\alpha \sum w_j^2$; shrinks but rarely zeros weights.

Subtopic	Meaning, example, interview angle
Alpha	Larger alpha means more bias, less variance; tune on CV.
Standardization	Essential: the penalty must treat feature scales fairly.

5. Algorithm / Working Process

Split, standardize features, select **alpha** with CV, minimize squared error plus L2 penalty, and apply the learned linear score at inference.

6. Mathematical Foundation

Minimize $\sum (y - Xw)^2 + \alpha \|w\|^2$. The solution is $(X^T X + \alpha I)^{-1} X^T y$. Adding αI makes the problem well-conditioned and works even when $p > n$.

7. Practical Implementation

```
from sklearn.linear_model import RidgeCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), RidgeCV(alphas=[.01, .1, 1, 10, 100]))
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(model[-1].alpha_)
```

8. Code Explanation

RidgeCV evaluates candidate penalty strengths using cross-validation. The pipeline ensures each fold learns scaling only from that fold's training portion.

9. Training / Evaluation

Search alpha logarithmically. Compare against OLS with CV RMSE; regularization is useful when test error improves, not merely because coefficients look smaller. Use nested CV for unbiased model selection estimates.

10. Complexity and Cost

Similar to linear regression; solvers vary by dense/sparse data. Prediction is $O(p)$ and CPU-friendly. Memory is dominated by X .

11. Common Use Cases

Correlated business indicators, genomics, TF-IDF regression, and regularized polynomial models.

12. Common Mistakes

Not scaling, tuning alpha on the test set, assuming Ridge performs feature selection, penalizing an intercept unintentionally, and interpreting shrunk coefficients as causal effects.

13. Edge Cases / Limitations

It does not remove irrelevant features, cannot capture nonlinear structure without features, and can underfit at excessive alpha.

14. Variations

Kernel Ridge captures nonlinearities; RidgeCV selects alpha; Bayesian ridge estimates regularization probabilistically. RidgeCV is placement-critical; kernel/Bayesian variants are useful extensions.

15. Related Topics

Lasso uses L1 and yields sparse weights; Elastic Net mixes both; PCA can reduce collinearity but loses direct feature semantics.

16. Interview Questions

1. **Ridge objective?** SSE plus $\alpha \sum w^2$.
2. **Why does it help?** It reduces variance from unstable coefficients.
3. **Does it set weights to zero?** Usually no.
4. **Why scale?** L2 penalizes numerical magnitude.
5. **Alpha zero?** OLS.
6. **Alpha infinity?** Non-intercept weights approach zero.
7. **Can p exceed n?** Yes.
8. **Bias-variance effect?** More bias, lower variance.
9. **Ridge vs Lasso with correlated features?** Ridge tends to keep/share them.
10. **How tune alpha?** Cross-validation.

17. Practice Tasks

Create correlated features; compare OLS/Ridge coefficient stability; tune alpha with CV; run polynomial Ridge; evaluate sparse TF-IDF data.

18. Project Ideas

Project	Stack/data/resume value
Retail-demand model	scikit-learn, sales data; stable correlated features.
Text score predictor	TF-IDF + Ridge; sparse high-dimensional ML.
Genomic response demo	synthetic wide data; $p > n$ reasoning.

19. Quick Revision

L2-shrunk OLS; $SSE + \alpha \|w\|^2$; use for collinearity; tune alpha; trap: unscaled data. **One-liner:** Ridge buys coefficient stability with controlled bias.

20. Final Cheat Sheet

Definition	I/O	Hyperparameter	Metrics	Pros / cons	Best use
L2-regularized linear regression	features → number	alpha	CV RMSE/MAE/R ²	stable; not sparse	correlated/wide inputs

Lasso Regression

1. Overview

Lasso adds an L1 penalty to linear regression, shrinking some coefficients exactly to zero. It is useful for embedded feature selection in high-dimensional tabular data.

2. Intuition

Give a model a limited "coefficient budget." Lasso often spends it on a small set of useful features and drops the rest.

3. Prerequisites

Linear regression, regularization, scaling, cross-validation, and sparse vectors.

4. Core Concepts

Subtopic	Meaning, example, interview angle
L1 penalty	$\alpha \sum$
Sparsity	Zero coefficient removes a feature from the linear score.
Correlation	Lasso may choose one of a correlated group arbitrarily; selection can be unstable.

5. Algorithm / Working Process

Standardize, choose alpha with CV, optimize the nonsmooth L1 objective (often coordinate descent), retain nonzero features, and predict with their weighted sum.

6. Mathematical Foundation

Minimize $\sum (y - Xw)^2 + \alpha \|w\|_1$. Absolute value is not differentiable at zero, so coordinate descent or proximal methods are used. Larger alpha produces more zeros.

7. Practical Implementation

```
from sklearn.linear_model import LassoCV
from sklearn.pipeline import make_pipeline
```

```
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), LassoCV(alphas=[.001, .01, .1, 1], cv=5,
max_iter=20_000))
model.fit(X_train, y_train)
print("selected:", (model[-1].coef_ != 0).sum())
```

8. Code Explanation

LassoCV selects alpha using five folds. The final line counts retained predictors; feature names should be paired with coefficients before making business claims.

9. Training / Evaluation

Scale features and tune alpha via CV. Measure test error, selection stability across resamples, and domain plausibility. Do not select features on all data before CV.

10. Complexity and Cost

Coordinate descent is iterative; cost depends on iterations and sparsity. Prediction is $O(k)$ for k nonzero weights, potentially cheaper than OLS.

11. Common Use Cases

Genomics, many engineered features, sparse text features, and interpretable compact scoring models.

12. Common Mistakes

No scaling, treating selected features as guaranteed important/causal, low **max_iter** nonconvergence, leakage in feature selection, and using it blindly with strongly correlated groups.

13. Edge Cases / Limitations

When useful features are correlated, it may discard an equally useful one. It can select at most roughly n features under common settings and underfits with high alpha.

14. Variations

LassoCV tunes alpha; group lasso selects feature groups; adaptive lasso reweights penalties. Standard Lasso is placement-critical; group/adaptive variants are research-aware extensions.

15. Related Topics

Ridge keeps correlated features; Elastic Net combines L1 and L2; recursive feature elimination is a separate wrapper-selection method.

16. Interview Questions

1. **Lasso objective?** SSE plus $\alpha ||w||_1$.
2. **Why zeros?** L1 geometry/proximal thresholding favors them.

3. **Why scale?** Otherwise penalty depends on units.
4. **Lasso vs Ridge?** Sparse selection vs stable shrinkage.
5. **How tune alpha?** CV.
6. **What with correlated features?** It may choose one unpredictably.
7. **Can alpha be zero?** It becomes OLS.
8. **Why convergence warning?** Too few iterations, scale issues, or difficult conditioning.
9. **Does zero prove irrelevance?** No.
10. **Alternative for correlated sparse selection?** Elastic Net.

17. Practice Tasks

Use a 1,000-feature dataset; plot coefficient paths over alpha; compare selected features across seeds; fix a convergence warning; compare Lasso and Elastic Net.

18. Project Ideas

Project	Stack/data/resume value
Gene-expression predictor	sklearn, public genomics; feature selection narrative.
Review-rating regression	TF-IDF + Lasso; sparse NLP baseline.
Churn-value estimator	customer data; concise, explainable feature set.

19. Quick Revision

L1 regularization selects features; $SSE + \alpha \|w\|_1$; use for sparse signal; CV RMSE; trap: correlated features.

One-liner: Lasso turns regularization into embedded feature selection.

20. Final Cheat Sheet

Definition	I/O	Hyperparameter	Pros / cons	Best use
L1-regularized linear regression	features → number	alpha	sparse/interpretable; unstable correlated selection	many mostly irrelevant features

Elastic Net

1. Overview

Elastic Net combines L1 and L2 penalties: it selects features like Lasso while stabilizing correlated groups like Ridge.

2. Intuition

Lasso is a strict budget and Ridge is a smooth weight restraint. Elastic Net uses both so related useful features can survive together.

3. Prerequisites

Ridge, Lasso, standardization, cross-validation, and bias-variance trade-off.

4. Core Concepts

Subtopic	Meaning, example, interview angle
<code>alpha</code>	Overall regularization strength.
<code>l1_ratio</code>	Mix: 1=Lasso, 0=Ridge; e.g. <code>.5</code> is balanced.
Grouping effect	L2 term encourages correlated features to have similar treatment.

5. Algorithm / Working Process

Standardize inputs, cross-validate `alpha` and `l1_ratio`, optimize with coordinate descent, then predict from retained/shrunk coefficients.

6. Mathematical Foundation

Minimize $SSE + \alpha[l1_ratio ||w||_1 + (1-l1_ratio) ||w||_2^2/2]$. The L1 part induces sparsity and L2 improves conditioning.

7. Practical Implementation

```
from sklearn.linear_model import ElasticNetCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), ElasticNetCV(l1_ratio=[.1, .5, .9, 1],
    alphas=[.001, .01, .1, 1], cv=5, max_iter=20_000)).fit(X_train, y_train)
print(model[-1].alpha_, model[-1].l1_ratio_)
```

8. Code Explanation

The grid includes Lasso (`l1_ratio=1`) as a candidate. Cross-validation chooses both the total penalty and how sparse versus stable the solution should be.

9. Training / Evaluation

Use scaled features, a log alpha grid, and a few meaningful ratios. Compare test performance and coefficient stability with Ridge/Lasso; a larger search is not automatically better.

10. Complexity and Cost

Iterative coordinate descent costs more than a single OLS solve, multiplied by CV candidates. Inference stays linear and CPU-friendly.

11. Common Use Cases

High-dimensional biology, correlated marketing indicators, text regression, and broad feature-engineering pipelines.

12. Common Mistakes

Confusing sklearn `alpha` with only L1 strength, forgetting scaling, tuning on test data, and assuming sparsity at a low `l1_ratio`.

13. Edge Cases / Limitations

Two hyperparameters add tuning cost; it still needs feature engineering for nonlinear patterns and can underfit under strong regularization.

14. Variations

Multi-task Elastic Net shares feature selection across targets; sparse-group Elastic Net adds group structure. Standard Elastic Net is interview-relevant; multi-task matters in research.

15. Related Topics

It bridges Ridge and Lasso. Group lasso uses known feature groups; PCA is another response to collinearity without sparse selection.

16. Interview Questions

1. **Objective?** L1 plus L2 penalized SSE.
2. **Why not only Lasso?** Correlated selections can be unstable.
3. **`l1_ratio=1`?** Lasso.
4. **`l1_ratio=0`?** Ridge-like, though use Ridge directly.
5. **Why scale?** Both penalties depend on coefficient magnitude.
6. **What does `alpha` control?** Total shrinkage.
7. **What does it select?** Some features can get zero weights.
8. **How tune?** CV over both parameters.
9. **Inference cost?** Linear in features/nonzeros.
10. **Best scenario?** Many correlated predictors with sparse signal.

17. Practice Tasks

Generate correlated blocks; compare coefficient paths; tune both parameters; test selection stability; try multi-output data.

18. Project Ideas

Project	Stack/data/resume value
Campaign-return forecast	sklearn, marketing data; correlated feature handling.
Molecular-property predictor	RDKit features + sklearn; high-dimensional regression.

Project	Stack/data/resume value
Article-engagement model	TF-IDF + Elastic Net; NLP tabular bridge.

19. Quick Revision

L1 + L2; tune alpha and **l1_ratio**; use for correlated sparse features; CV RMSE; trap: no scaling. **One-liner:** Elastic Net is Lasso made less brittle by Ridge.

20. Final Cheat Sheet

Definition	I/O	Key hyperparameters	Pros / cons	Best use
Mixed L1/L2 linear regression	features → number	alpha, l1_ratio	sparse and stable; extra tuning	correlated, high-dimensional data

Logistic Regression

1. Overview

Despite its name, logistic regression is a linear **classification** model that predicts class probabilities. It is widely used for fraud, churn, triage, and calibrated binary baselines.

2. Intuition

It turns a linear score into a number between 0 and 1 using an S-shaped sigmoid, then applies a decision threshold.

3. Prerequisites

Linear algebra, probability, odds/log-odds, classification metrics, gradient optimization, and regularization.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Sigmoid	$\sigma(z)=1/(1+e^{-z})$ maps score to probability.
Log-odds	$\log(p/(1-p))=b+w^T x$; a coefficient changes odds multiplicatively.
Threshold	.5 is not universal; choose from business cost/recall needs.
Regularization	L2 default; L1 can select features.

5. Algorithm / Working Process

Input features and class labels. Encode/impute/scale, compute a linear logit, convert it to probability, minimize log loss during training, and threshold probabilities or return rankings at inference.

6. Mathematical Foundation

$p(y=1|x)=\sigma(w^T x+b)$. Binary cross-entropy is $-[y \log p+(1-y)\log(1-p)]$; maximum likelihood of Bernoulli labels gives the same objective. Add L1/L2 penalties for regularization.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, stratify=y, test_size=.2,
random_state=42)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2_000,
class_weight="balanced")).fit(Xtr, ytr)
p = model.predict_proba(Xte)[:, 1]
print("ROC-AUC:", roc_auc_score(yte, p)); print(classification_report(yte, p >=
.5))
```

8. Code Explanation

Stratification preserves class proportions. `predict_proba` gives scores for ROC-AUC; the `.5` cutoff produces labels for precision/recall/F1. `class_weight="balanced"` is useful when classes are imbalanced, but verify its impact.

9. Training / Evaluation

Use stratified CV. Evaluate ROC-AUC for ranking, PR-AUC for rare positives, and precision/recall based on error costs; inspect calibration (Brier score/reliability curve). Tune `C` (inverse regularization) and threshold separately.

10. Complexity and Cost

Iterative training is roughly proportional to $n \times p \times \text{iterations}$; inference is $O(p)$ per class. CPU is sufficient for standard tabular/sparse problems.

11. Common Use Cases

Spam/fraud probability, medical screening, customer churn, credit risk, click-through baseline, and multiclass document classification.

12. Common Mistakes

Calling it regression for continuous values, using accuracy alone on imbalance, thresholding before AUC, leakage in scaling, unscaled features with regularization, and interpreting odds ratio as probability change.

13. Edge Cases / Limitations

Linear decision boundary unless features are engineered; complete separation can cause unstable unregularized estimates; probabilities may be miscalibrated under distribution shift.

14. Variations

Multinomial softmax handles mutually exclusive classes; one-vs-rest handles multi-label style classification; ordinal logistic models ordered classes. Binary/multinomial are placement essentials.

15. Related Topics

Linear regression predicts unbounded numeric output; SVM optimizes margin rather than likelihood; Naive Bayes uses a generative probability model; calibrated trees can offer nonlinear probabilities.

16. Interview Questions

1. **Why “logistic”?** It uses the logistic sigmoid.
2. **Output?** A class probability, then optional label.
3. **Loss?** Binary cross-entropy/log loss.
4. **Why not MSE?** Log loss matches Bernoulli likelihood and gives better gradients.
5. **Coefficient interpretation?** One-unit increase changes log-odds by the coefficient.
6. **What is C in sklearn?** Inverse regularization strength.
7. **Accuracy issue?** It hides failure on rare classes.
8. **ROC-AUC vs PR-AUC?** PR-AUC is more informative for rare positives.
9. **How set threshold?** Optimize for business costs/target recall or precision.
10. **Multiclass method?** Softmax/multinomial or one-vs-rest.

17. Practice Tasks

Train a churn classifier; compare thresholds by confusion matrix; plot ROC and PR curves; calibrate probabilities; add L1 feature selection.

18. Project Ideas

Project	Stack/data/resume value
Loan-default screener	sklearn, credit-risk data; threshold/cost discussion.
Cancer classifier	Wisconsin dataset; metrics and calibration.
Spam detector	TF-IDF + logistic regression; strong NLP baseline.

19. Quick Revision

Classifier with sigmoid; $p = \sigma(\mathbf{w}^T \mathbf{x})$ and log loss; use for probability/ranking; ROC-AUC, PR-AUC, F1; trap: accuracy and fixed .5 threshold. **One-liner:** it models log-odds as a linear function of features.

20. Final Cheat Sheet

Definition	I/O	Key hyperparameters	Metrics	Pros / cons	Best use
Linear probabilistic classifier	features → class probability	C, penalty, threshold	ROC/PR-AUC, precision, recall	fast/calibratable; linear boundary	tabular/sparse classification

Robust Regression

1. Overview

Robust regression estimates a relationship without letting a few outliers dominate it. It matters for sensor noise, financial anomalies, and contaminated operational data.

2. Intuition

OLS lets one wildly wrong measurement pull the whole line. Robust losses reduce that point's influence while still fitting normal observations.

3. Prerequisites

OLS, residuals, outliers versus leverage points, loss functions, and robust statistics.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Huber loss	Quadratic near zero, linear for large residuals; balances OLS and MAE.
RANSAC	Fits many random subsets, retaining the model supported by inliers.
Leverage	Extreme X values can be harmful even with a robust residual loss.

5. Algorithm / Working Process

For Huber: initialize a fit, downweight large residuals, refit iteratively, and predict linearly. For RANSAC: sample minimal subsets, fit candidates, count inliers, then refit on consensus points.

6. Mathematical Foundation

Huber loss is $.5r^2$ when $|r| \leq \delta$ and $\delta(|r| - .5\delta)$ otherwise. It has OLS-like efficiency for small noise and MAE-like resistance to large residuals. RANSAC optimizes inlier consensus, not a smooth global loss.

7. Practical Implementation

```
from sklearn.linear_model import HuberRegressor, RANSACRegressor, LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
huber = make_pipeline(StandardScaler(), HuberRegressor(epsilon=1.35,  
max_iter=1_000)).fit(X_train, y_train)  
ransac = RANSACRegressor(estimator=LinearRegression(),  
random_state=42).fit(X_train, y_train)  
pred = huber.predict(X_test)
```

8. Code Explanation

Huber softens the effect of large residuals; **epsilon** controls where the loss changes shape. RANSAC is preferable when a clear inlier majority exists and outliers may be extreme.

9. Training / Evaluation

Use clean test labels if possible; report MAE and median absolute error alongside RMSE. Diagnose outliers before deleting data—an outlier might be an important rare event. Tune Huber epsilon or RANSAC residual threshold using validation/domain noise scale.

10. Complexity and Cost

Huber is iterative. RANSAC cost scales with number of trials and can grow sharply when inlier fraction falls. Both usually run on CPU.

11. Common Use Cases

Sensor calibration, geospatial line fitting, transaction-value prediction with errors, and lab measurements.

12. Common Mistakes

Removing points blindly, using RANSAC where anomalies are the target, assuming robust loss fixes leverage points, and scoring only with RMSE when outlier resilience is the goal.

13. Edge Cases / Limitations

RANSAC fails with too few inliers; robust models can ignore meaningful extreme regimes; neither fixes missing features or a nonlinear true relation.

14. Variations

Theil–Sen uses median slopes and is highly robust for small dimensions; Tukey loss redescends; robust scaling helps preprocessing. Huber/RANSAC are the key placement variants.

15. Related Topics

Quantile regression describes asymmetric outcomes; L1/MAE is robust but less smooth; anomaly detection decides whether unusual points should be modeled separately.

16. Interview Questions

1. **Why OLS is outlier-sensitive?** Squaring makes large residuals dominate.

2. **Huber behavior?** L2 near zero, L1 for large residuals.
3. **What does RANSAC seek?** Largest consensus/inlier set.
4. **Outlier vs leverage point?** Unusual y versus unusual X.
5. **Does robust mean ignore all extremes?** No; validate their meaning.
6. **When RANSAC?** Gross outliers with a majority inlier structure.
7. **When Huber?** Moderate contamination/noisy residuals.
8. **Why median absolute error?** It is robust for evaluation.
9. **Can it fit nonlinear data?** Only after nonlinear features/model changes.
10. **RANSAC weakness?** Low inlier fraction and threshold sensitivity.

17. Practice Tasks

Inject 5% outliers and compare OLS/Huber/RANSAC; plot residuals; vary RANSAC thresholds; identify leverage points; measure clean-test MAE.

18. Project Ideas

Project	Stack/data/resume value
Sensor-calibration service	sklearn, sensor logs; noise-aware modeling.
Road-lane line estimator	OpenCV + RANSAC; classic CV geometry.
Property-price cleaner	tabular data; explains anomaly policy.

19. Quick Revision

Outlier-resistant fitting; Huber loss; use for contaminated labels; median AE/MAE; trap: leverage points. **One-liner:** robust regression limits the influence of abnormal residuals.

20. Final Cheat Sheet

Definition	I/O	Hyperparameters	Pros / cons	Best use
Outlier-resistant regression	features → number	epsilon, residual threshold/trials	robust; needs outlier policy	noisy sensor/measurement data

Quantile Regression

1. Overview

Quantile regression predicts a conditional percentile, not just the conditional mean. It is valuable when uncertainty or asymmetric costs matter: delivery ETA P90, risk P95, or low-end wage estimates.

2. Intuition

Mean prediction answers "typical value." A 90th-quantile model answers "a value that 90% of outcomes should not exceed."

3. Prerequisites

Percentiles, conditional distributions, residuals, linear regression, and asymmetric costs.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Quantile q	$q=.5$ is the median; $.9$ targets the 90th percentile.
Pinball loss	Penalizes underprediction and overprediction asymmetrically.
Prediction interval	Fit $.05$ and $.95$ quantiles for an empirical interval.

5. Algorithm / Working Process

Choose one or more target quantiles, fit a model minimizing pinball loss for each, then return the requested percentile(s) for a new input. Different quantiles may have different slopes.

6. Mathematical Foundation

With residual $r=y-y_{\text{hat}}$, pinball loss is $\max(qr, (q-1)r)$. At $q=.5$, it becomes proportional to absolute error and estimates the median. A calibrated q -quantile has about q fraction of observations below it.

7. Practical Implementation

```
from sklearn.linear_model import QuantileRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

q90 = make_pipeline(StandardScaler(), QuantileRegressor(quantile=.90, alpha=.01,
solver="highs"))
q90.fit(X_train, y_train)
p90_eta = q90.predict(X_test)
coverage = (y_test <= p90_eta).mean()
print("empirical P90 coverage:", coverage)
```

8. Code Explanation

`quantile=.90` changes the target from mean to upper-tail estimate. Coverage checks calibration: it should be near $.90$ on representative unseen data, not necessarily exact on a small test set.

9. Training / Evaluation

Use pinball loss at each business quantile and empirical coverage. For intervals, check both coverage and width. Tune regularization; use time-aware validation for ETAs/forecasting. Quantile crossing ($P90 < P50$) needs monitoring.

10. Complexity and Cost

Linear quantile regression is commonly solved by linear programming and may be slower than OLS. One model per quantile increases fit and inference cost linearly in number of quantiles.

11. Common Use Cases

P90 delivery time, value-at-risk style estimates, insurance reserves, capacity planning, and fairness/wage distribution analysis.

12. Common Mistakes

Calling a quantile a probability for an individual event, evaluating only RMSE, ignoring quantile crossing, using mean-target assumptions, and setting q without business rationale.

13. Edge Cases / Limitations

Tail quantiles need enough tail data; linear quantiles miss nonlinear conditional distributions; independently fit quantiles can cross.

14. Variations

Median regression uses $q=.5$; quantile random forests/gradient boosting model nonlinear quantiles; conformal prediction adds distribution-free coverage ideas. Median/P90 are placement-relevant; conformal is a strong advanced topic.

15. Related Topics

Robust regression protects the central fit; quantile regression intentionally targets a different part of the outcome distribution. Gaussian processes provide a full predictive distribution under assumptions.

16. Interview Questions

1. **What does $q=.9$ predict?** Conditional 90th percentile.
2. **$q=.5$ equals?** Conditional median.
3. **Loss?** Pinball/check loss.
4. **Why not RMSE?** It evaluates mean error, not quantile calibration.
5. **Why ETA P90?** Operational promises need conservative tail estimates.
6. **Underprediction cost at $q=.9$?** Nine times the overprediction slope.
7. **What is coverage?** Fraction of actual values below predicted quantile.
8. **What is quantile crossing?** Higher estimated quantile below a lower one.
9. **Can slopes differ by quantile?** Yes.
10. **Alternative for nonlinear quantiles?** Quantile gradient boosting/forests.

17. Practice Tasks

Fit P10/P50/P90; calculate coverage and width; plot crossings; compare median versus mean with skewed errors; make an ETA SLA simulator.

18. Project Ideas

Project	Stack/data/resume value
Delivery-SLA predictor	sklearn, delivery data; uncertainty-aware product decision.
Solar-generation bands	weather + energy data; interval forecasts.
Rent-range estimator	housing data; gives users useful ranges, not one point.

19. Quick Revision

Predicts conditional percentile; pinball loss; use for tails/intervals; coverage + pinball loss; trap: RMSE and crossing. **One-liner:** it models "how bad could it be at P90?" rather than the average.

20. Final Cheat Sheet

Definition	I/O	Key hyperparameters	Metrics	Pros / cons	Best use
Conditional percentile regressor	features → q-th value	quantile, alpha	pinball loss, coverage	tail-aware; multiple models/crossing	SLAs, uncertainty bands

Bayesian Linear Regression

1. Overview

Bayesian linear regression places probability distributions over coefficients and predictions. It produces both a mean prediction and uncertainty, useful with small data or high-stakes decisions.

2. Intuition

Instead of declaring one "true" slope, start with plausible slopes, update beliefs using data, and keep uncertainty where evidence is weak.

3. Prerequisites

Linear regression, Gaussian distributions, Bayes' rule, likelihood/prior/posterior, and variance.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Prior	Belief about weights before data, e.g. small weights are likely.
Likelihood	Probability of observed targets given weights/noise.
Posterior	Updated weight distribution after combining prior and data.
Predictive uncertainty	Epistemic uncertainty from weights plus observation noise.

5. Algorithm / Working Process

Specify Gaussian noise and a prior over weights, update to a posterior using training data, integrate over posterior weights to obtain a predictive mean and standard deviation for each new input.

6. Mathematical Foundation

Assume $y|X, w \sim N(Xw, \beta^{-1} I)$ and $w \sim N(0, \alpha^{-1} I)$. Then posterior covariance is $S_N^{-1} = \alpha I + \beta X^T X$; posterior mean is $m_N = \beta S_N^{-1} y$. Predictive variance includes $x^T S_N x + \beta^{-1}$. A Gaussian prior resembles Ridge regularization (MAP estimate).

7. Practical Implementation

```
from sklearn.linear_model import BayesianRidge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), BayesianRidge()).fit(X_train, y_train)
mean, std = model.predict(X_test, return_std=True)
lower, upper = mean - 1.96 * std, mean + 1.96 * std
print(list(zip(mean[:3], lower[:3], upper[:3])))
```

8. Code Explanation

`BayesianRidge` estimates precision-related parameters empirically. `return_std=True` exposes predictive standard deviation; the 1.96 interval is approximately 95% only when model assumptions/calibration are reasonable.

9. Training / Evaluation

Evaluate MAE/RMSE plus interval coverage and sharpness. Standardize features, validate uncertainty on held-out data, and distinguish uncertainty from a wide irreducible-noise process. Informative priors require domain justification.

10. Complexity and Cost

Exact dense fitting has matrix costs similar to Ridge, approximately cubic in feature count. Prediction mean is linear in features; variance adds quadratic-form work. CPU is typical.

11. Common Use Cases

Scientific measurements, low-data forecasting, risk-aware estimates, online experiments, and interpretable uncertainty-aware baselines.

12. Common Mistakes

Treating standard deviation as guaranteed confidence, ignoring distribution shift, confusing posterior uncertainty with prediction error, using arbitrary priors, and skipping coverage checks.

13. Edge Cases / Limitations

Gaussian/noise linearity assumptions may be wrong; exact inference becomes costly for many features; uncertainty can be overconfident under misspecification.

14. Variations

Bayesian Ridge is empirical-Bayes practical; ARD learns per-feature precisions and can prune; full Bayesian regression uses MCMC/variational inference. Bayesian Ridge/ARD are good placement extensions; MCMC is research depth.

15. Related Topics

Ridge is the MAP cousin; Gaussian processes are Bayesian regression with a distribution over functions; conformal prediction offers coverage without a Bayesian model assumption.

16. Interview Questions

1. **What is a prior?** Belief before observing data.
2. **Posterior?** Belief after prior and likelihood combine.
3. **Why Bayesian regression?** Point predictions plus uncertainty.
4. **MAP relation to Ridge?** Gaussian prior gives L2 penalty.
5. **Epistemic uncertainty?** Lack of knowledge, reduced by data.
6. **Aleatoric uncertainty?** Inherent observation noise.
7. **Why predictive variance has two terms?** Parameter uncertainty and noise.
8. **Is a 95% interval always 95% coverage?** Only if calibrated/assumptions hold.
9. **What is ARD?** Per-feature prior precision learning.
10. **Main limitation?** Misspecified assumptions can create false confidence.

17. Practice Tasks

Compare Ridge and BayesianRidge; plot uncertainty with increasing data; measure 95% coverage; change feature range to see epistemic uncertainty; test ARD.

18. Project Ideas

Project	Stack/data/resume value
Lab-measurement predictor	sklearn, calibration data; uncertainty reporting.
Small-data price bands	BayesianRidge, housing sample; risk-aware predictions.
Experiment-effect estimator	NumPy/sklearn, A/B data; probabilistic reasoning.

19. Quick Revision

Distribution over weights; Gaussian prior + likelihood; use when uncertainty matters; RMSE + coverage; trap: trusting uncalibrated intervals. **One-liner:** Bayesian linear regression predicts and says how uncertain it is.

20. Final Cheat Sheet

Definition	I/O	Key assumptions	Metrics	Pros / cons	Best use
Bayesian weighted-sum model	features → mean and std	linear Gaussian noise/prior	RMSE, coverage	uncertainty-aware; assumption-sensitive	small-data risk-aware prediction

Gaussian Process Regression

1. Overview

Gaussian process regression (GPR) is a nonparametric Bayesian model that defines a distribution over smooth functions. It excels on small, expensive datasets where uncertainty and flexible curves matter, such as Bayesian optimization and scientific modeling.

2. Intuition

Rather than choose a fixed curve shape, define how similar two inputs are. Nearby inputs should have correlated outputs; observed points bend the distribution of possible curves and uncertainty falls nearby.

3. Prerequisites

Multivariate Gaussians, covariance matrices, kernels, Bayes intuition, matrix factorization, and feature scaling.

4. Core Concepts

Subtopic	Meaning, example, interview angle
Kernel	Similarity function; RBF makes nearby points strongly correlated.
Length scale	How quickly function changes; small means wiggly, large means smooth.
Noise term	Observation variance, commonly <code>WhiteKernel</code> or <code>alpha</code> .
Posterior	Mean interpolates patterns; variance grows far from data.

5. Algorithm / Working Process

Standardize inputs/target, choose a kernel and noise model, form the training covariance matrix, optimize kernel hyperparameters by log marginal likelihood, factorize the matrix, and return predictive mean/std for new inputs.

6. Mathematical Foundation

For training covariance K and noise $\sigma_n^2 I$, predictive mean is $k_*^T (K + \sigma_n^2 I)^{-1} y$; predictive variance is $k(x^*, x^*) - k_*^T (K + \sigma_n^2 I)^{-1} k_*$. The RBF kernel is $k(x, x') = \sigma_f^2 \exp(-||x - x'||^2 / (2l^2))$.

7. Practical Implementation

```

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel, ConstantKernel
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

kernel = ConstantKernel() * RBF(length_scale=1.0) + WhiteKernel(noise_level=.1)
model = make_pipeline(StandardScaler(), GaussianProcessRegressor(kernel=kernel,
normalize_y=True,
    n_restarts_optimizer=5, random_state=42)).fit(X_train, y_train)
mean, std = model.predict(X_test, return_std=True)
print(model[-1].kernel_)

```

8. Code Explanation

The RBF component learns smooth correlation and `WhiteKernel` models measurement noise. Restarts reduce the chance of a poor local hyperparameter optimum. `return_std` is GPR's principal practical advantage.

9. Training / Evaluation

Scale inputs, select kernel from domain knowledge, and validate RMSE/MAE plus uncertainty coverage and negative log predictive density. Inspect learned length scales; a boundary value may signal a poor kernel/search range. Use a held-out set—not only marginal likelihood.

10. Complexity and Cost

Exact GPR training stores an $n \times n$ covariance matrix: $O(n^3)$ time and $O(n^2)$ memory; prediction costs about $O(n)$ mean and $O(n^2)$ variance per batch implementation. Usually CPU-only but impractical for large n (often tens of thousands or less depending on hardware).

11. Common Use Cases

Bayesian optimization, robot/sensor calibration, surrogate modeling for simulations, geospatial interpolation, and small scientific datasets.

12. Common Mistakes

Skipping scaling, using GPR on millions of rows, treating its standard deviation as universally calibrated, omitting noise for noisy observations, and choosing an RBF kernel without considering periodicity/linear trends.

13. Edge Cases / Limitations

It scales poorly with samples, kernel choice dominates results, high-dimensional distances become less informative, and extrapolation generally reverts toward the prior mean rather than continuing trends.

14. Variations

Matérn kernels model rougher functions; periodic kernels capture cycles; additive/product kernels encode structure; sparse/variational GPs scale larger data. RBF/Matérn and complexity are placement essentials;

sparse GPs are research/production depth.

15. Related Topics

Bayesian linear regression is a GP with a linear kernel; kernel ridge provides similar kernelized predictions without the full probabilistic interpretation; Bayesian optimization uses a GP surrogate plus acquisition function.

16. Interview Questions

1. **What does a GP model?** A distribution over functions.
2. **What is a kernel?** A covariance/similarity function.
3. **RBF length scale meaning?** Distance over which outputs remain correlated.
4. **Why uncertainty grows away from data?** The prior dominates where evidence is absent.
5. **Training complexity?** $O(n^3)$.
6. **Memory complexity?** $O(n^2)$.
7. **Why scale inputs?** Kernel distances and length scales depend on units.
8. **What handles noisy observations?** Noise variance/WhiteKernel.
9. **GP vs Bayesian linear regression?** Flexible kernel functions versus linear basis.
10. **Why useful in Bayesian optimization?** It supplies prediction and uncertainty for exploration.

17. Practice Tasks

Fit noisy sine data; compare RBF and Matérn; plot mean $\pm$ 2 std; vary length scale; measure time as sample count grows; use a periodic kernel on seasonal data.

18. Project Ideas

Project	Stack/data/resume value
Hyperparameter tuner	scikit-learn GPR; demonstrates Bayesian optimization concepts.
Air-quality interpolator	geospatial sensor data; uncertainty maps.
Materials surrogate	simulation/chemistry data; expensive-experiment modeling.

19. Quick Revision

Bayesian distribution over functions; kernel covariance; use for small-data nonlinear uncertainty; RMSE + coverage/NLPD; trap: $O(n^3)$ scaling. **One-liner:** a GP predicts from similarity to observed points and quantifies ignorance.

20. Final Cheat Sheet

Definition	I/O	Key hyperparameters	Metrics	Pros / cons	Best use
Kernel-based Bayesian function model	features $\rightarrow$ mean and std	kernel, length scale, noise	RMSE, coverage, NLPD	flexible/uncertain; $O(n^3)$ train	small expensive

Definition	I/O	Key hyperparameters	Metrics	Pros / cons	Best use
					nonlinear data